

Experiment 4: Binary Search with Big-O Analysis

Aim

To write a Python program to **search for an element in a sorted array using Binary Search** and analyze its Big-O time complexity.

Algorithm

1. Start.
2. Read a sorted array and the element to be searched.
3. Set $low = 0$ and $high = n - 1$.
4. Find the middle element using $mid = (low + high) // 2$.
5. If the middle element is equal to the search element, return its position.
6. If the search element is greater than the middle element, search the right half.
7. Otherwise, search the left half.
8. Repeat until the element is found or $low > high$.
9. Display the result.
10. Stop.

Source Code

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

arr = [10, 20, 30, 40, 50, 60, 70]
target = 50

result = binary_search(arr, target)

print("Array:", arr)
print("Element to Search:", target)

if result != -1:
    print("Element found at index:", result)
else:
```

```
print("Element not found")
```

Output :

Output

```
Array: [10, 20, 30, 40, 50, 60, 70]  
Element to Search: 50  
Element found at index: 4
```

Result :

Thus, the Python program was successfully executed and the element **50** was successfully found at **index 4** using Binary Search.